

Arteamis — Product Requirements Document

Product	Arteamis — a governed Company Brain for AI-native studios
Scope	The Governed Studio Brain wedge (a deliberately narrow slice of a broader long-term vision, which is out of scope here)
Status	Pre-validation. Core system built and tested ; not yet validated with real customers
Last updated	July 2026

Reading the tags. Every requirement and claim is tagged **[built]** (exists and is tested today), **[hypothesis]** (believed, not yet tested with a real user), or **[validated]** (confirmed by real customer evidence). Right now almost nothing is **[validated]** — and stating that plainly is deliberate. This document is written to be *updated by evidence from real users*, not rewritten by the next idea. A short sources list is at the end.

1. Summary

You already know the problem from your own week: you're deep in a Claude Code or Cursor session, and the agent doesn't know this repo's conventions, this client's rules, or the decision you made three sprints ago about why you don't do it the obvious way. So you re-explain it. Again. And when the agent hands something back, you can't tell at a glance whether it followed the rules or confidently made something up.

Arteamis fixes that. It's a **governed Company Brain**: it turns a studio's scattered knowledge — repo conventions, client rules, security history, past decisions — into reviewed, source-backed memory, and hands that memory to your agents as scoped context they're allowed to use. "Governed" is the load-bearing word: nothing becomes team truth, and nothing reaches an agent, without a human review step and an audit trail.

The wedge — the one narrow thing we sell first:

Governed Studio Brain + Agent Context Pack — so agentic software / security / Web3 studios stop re-explaining context to Claude Code and Cursor, and can trust what those agents hand back.

Why a competitor can't just clone it. The obvious stuff is already commoditizing — skills, the Model Context Protocol (MCP), and agent memory are turning into free platform primitives (Anthropic's Agent Skills became an open standard that ~40 tools adopted within months). Building "connect your docs, answer with citations, expose MCP" buys you nothing durable; everyone will have it. The part that *doesn't* commoditize is a specific company's **governed context** — what's true here, what's allowed,

what an agent may read, whether its output earned trust — plus the record that piles up as you use it: *what context went in → what the agent did → what a human decided → what actually happened*. You only get that record by running real governed work inside real companies over real time. That's the thing that's hard to copy, and it's what we're building toward owning.

Where this actually stands. The engine is real and tested — the governed loop runs end-to-end on real infrastructure and passes the security evals (§3). What is **not** proven yet: that a studio will pay for it and feel it save time. That's the whole job of the next phase, and this doc is deliberate about which claims are already true versus which ones we still have to go prove.

2. How Arteamis is shaped

Before the details, the one mental model that makes everything else click. If you hold this picture, the rest of the doc is easy.

Two kinds of memory, one bridge. Every person has a **Personal Brain** — private by default, their own notes, sources, and rough thinking. The studio has a **Company Brain** — shared, reviewed, permissioned truth. The only way something moves from private to shared is **promotion**: a deliberate, human-reviewed, audited step. Nothing crosses silently.

```
graph LR; A["Personal Brain (private)  
your raw sources,  
notes, private Q&A"] -- "promotion (reviewed, audited)" --> B["Company Brain (shared)  
what the team has  
agreed is true"]
```

Personal Brain (private) —promotion (reviewed, audited)—> Company Brain (shared)
your raw sources, what the team has
notes, private Q&A agreed is true

Two surfaces — but only one exists today. Be precise about this:

- **Web app — the team surface. [built, this is what runs now].** The shared Company Brain: reviewed knowledge, team rules, decisions, governed work handoffs, agent context, audit. This is the product we're building and selling in the wedge.
- **Desktop app — the private/local surface. [planned, not built].** A local-first Personal Brain and "Local Vault" so a person's raw sources — client code, secrets, unfinished thinking — can stay **on their own machine and never touch the cloud**, while only reviewed, redacted, promoted knowledge goes up to the web Company Brain. The architecture is decided; the app is not built yet, and it's a non-goal for the current wedge.

The reason for the split isn't "an app for individuals vs. an app for teams" — it's the **privacy/trust boundary**. A security-minded studio's biggest fear is raw client code or secrets leaking into a shared system. The hybrid shape answers that directly: private and local stays private and local; only what's explicitly reviewed becomes shared. Today that boundary is enforced inside the web app (personal vs. company, per-user isolation); the desktop app later makes "never leaves my machine" literally true.

(Note on "productivity suite": a full suite — chat, docs, tasks, meetings — is part of the long-term vision, not the wedge. Today the web app is the governed brain loop, not a Slack/Notion replacement.)

3. Current state — what exists today

So the reader can gauge reality, not just intent. Detail is deliberately high-level (no code).

The backend is real and well-tested. A governed maker/checker loop runs the full chain — private capture → reviewed shared lesson → team rule → agent brief → work handoff → agent-work check → learning review — over real infrastructure (FastAPI, PostgreSQL + vector search), with per-request tenant isolation and an immutable audit log. Scale of what's there: ~47 API modules and ~100 automated test files. **[built]**

Verified working: source ingestion (PDF/DOCX/text/URL) with grounded, **cited** answers that refuse when unsupported; the governed promotion loop with a human review gate; and a security-evaluation suite in which all five abuse categories — tenant leakage, prompt injection, indirect injection, memory poisoning, excessive agency — fail closed with a controlled error and an audit event, zero silent failures. The full loop has also been run end-to-end against real model APIs (no mocks) — capture through learning review, with cost-per-loop recorded. **[built] / [validated] on the security evals**

The frontend is a thin client wired to that backend for the core loop, with a simplified first-time-user surface (a beginner sees ~6 objects; advanced machinery is hidden, not renamed). Deployment (cloud infra, encrypted secrets, CI/CD) is in place. **[built]**

What is deliberately NOT built yet (the long-term vision, out of this wedge's scope): a full productivity suite, AI-native company formation, decision- intelligence BI, company-operating-system primitives, a native desktop app, and graph-governance tooling. These are real future directions, intentionally parked.

What is genuinely unproven: that a real studio will adopt, pay, and measurably save time. No customer validation has happened yet. Everything user- and market-facing below is therefore a **[hypothesis]** .

4. The problem

AI coding agents are now good enough to do real studio work. The blocker is no longer the model — it's **context and trust**.

A studio's operating knowledge — codebase conventions, client-specific delivery rules, security and audit history, on-chain assumptions, prior decisions, reusable patterns, known mistakes, review rules — lives in people's heads, chat threads, repos, and scattered docs. The consequences the buyer feels every week: **[hypothesis]**

- **Repeated explanation.** Senior people re-explain the same context to teammates and to agents, over and over. The founder becomes a human context server.
- **Agents start from zero.** Every Claude Code / Cursor session re-learns the project; the studio's hard-won conventions don't carry from one task or project to the next.

- **No trust boundary on agent output.** Confident-but-wrong agent work looks the same as correct work. Nothing separates "reviewed and safe to reuse" from "a plausible guess."
- **Knowledge doesn't compound.** What one engineer learns debugging an agent failure is lost; the next person (or agent) repeats the mistake.

Two failure modes Artemis refuses

- **Dump-and-trust.** Upload documents → let AI summarize → trust the output → compound wrong knowledge. Artemis is a *second* brain only after a human's *first* brain has read, verified, and asserted. **[built]** — enforced by the review gate, not merely stated.
 - **The ungoverned brain.** Memory that silently becomes "company truth" with no source, no review, no audit — the path to a poisoned brain. Artemis makes promotion explicit and audited. **[built]**
-

5. Users & personas

Five user types, including one that isn't human. The thing to get right: they don't all want the same amount of AI doing things for them. An engineer wants to hand off routine work and move on; a reviewer wants their hand on every gate; the founder wants tight control on strategy but happily delegates once context is trusted. So "how much autonomy" is a per-user, per-task setting, not a global one. (There's real research behind this — the "human agency scale," Stanford 2026 — but the point is just: don't force one autonomy level on everyone.)

(A broader vision adds more personas — product lead, designer, BD/marketing, operators. Those are out of scope for this wedge.)

5.1 The Founder / CTO — buyer & power user

- **Who:** technical founder or CTO of a 1–50-person agentic studio, already running Claude Code / Cursor daily.
- **Jobs-to-be-done:** stop being the human context server; get leverage from agents without losing control; keep the studio's judgment when people or agents change.
- **Pains:** carries too much in their head; asked "why?" constantly; watches agents redo solved problems; can't safely delegate high-stakes work.
- **Success:** measurably fewer repeated explanations; agents that start with the studio's real context; a record of decisions that outlives any one person.
- **Agency level:** augmentation with **high control** on strategy and security; happy to delegate execution once context is trusted.

5.2 The Engineer — the maker

- **Who:** an engineer shipping client/product work with heavy agent assistance.
- **Jobs-to-be-done:** get project context without interrupting a teammate; make agents follow the studio's conventions; not repeat known mistakes.

- **Pains:** re-explains the codebase to every new agent session; hits the same failure twice; unsure which past decision still holds.
- **Success:** one place to ask the project privately and get a cited answer; agent context packs that already know the conventions; do-not-repeat warnings.
- **Agency level: high delegation** on routine execution, with a fast path to inspect why an answer or agent action happened.

5.3 The Reviewer / Lead — the checker

- **Who:** a tech lead or delivery lead who owns what becomes team truth.
- **Jobs-to-be-done:** gatekeep promotions into shared memory; verify source and impact before a claim becomes a rule; keep an audit trail.
- **Pains:** knowledge gets shared with no proof; can't tell AI-drafted from human-verified; no record of who approved what and why.
- **Success:** every proposed team update shows its source, what changed, and trust level; accept/reject is one reviewed action; nothing mutates canonical memory without them.
- **Agency level: maximum control** — this is the human-judgment gate the whole product depends on.

5.4 The AI Agent — a first-class non-human user

- **Who:** Claude Code, Cursor, an MCP client, or a future worker runtime.
- **Jobs-to-be-done:** receive *scoped, governed* context for a task; know what it may and may not read; return a trace of what it did.
- **Pains (its human owner's pains):** agents over-read (leak) or under-read (miss the convention); their output can't be trusted or audited.
- **Success:** an Agent Context Pack with allowed context, denied context (with reasons), the relevant skill, and constraints; risky writes are propose-only; every read/action is logged.
- **Agency level: bounded autonomy** — acts freely within an approved contract, stops at the gate for high-impact writes.

5.5 The New Hire — the context consumer

- **Who:** someone joining the studio or moving onto a new project.
 - **Jobs-to-be-done:** understand the project's *why* without draining the founder's time.
 - **Pains:** onboarding is tribal knowledge and interruptions.
 - **Success:** current beliefs, decisions, and first tasks with sources, on day one; questions cite proof; onboarding gaps become brain improvements.
 - **Agency level:** augmentation — learn and verify, not blindly trust.
-

6. Goals & non-goals

Goals (this wedge)

1. A studio can turn its real knowledge (repos, docs, client notes, security rules, decisions) into a **reviewed, source-backed Studio Brain**. **[built]**
2. That brain produces **governed Agent Context Packs** the studio's agents actually use. **[built]**
3. Every promotion into team memory is **human-reviewed, source-backed, and audited**. **[built]**
4. The studio can **measure** the value: less repeated explanation, less redone agent work, visible cost per useful loop. **[built] instrumentation; [hypothesis] that the numbers move.**
5. A first-time user can **complete the loop without the founder explaining it**. **[hypothesis]** — the simplified surface exists **[built]**, but comprehension is unproven with real users.

Non-goals (deliberately deferred)

Full productivity suite · AI-native company formation · decision-intelligence BI · company-OS primitives · graph-governance UI · every-member personal brain · native desktop app · Slack/Notion/Jira/Lark replacement · autonomous agents that act without approval.

7. Solution & the core loop

Every member has a **Personal Brain** (private by default). The studio has a **Company Brain** (shared, reviewed, permissioned). The bridge is **intentional promotion** — nothing crosses silently.

A first-time user meets **six objects**, not the full model:

source · private answer · share proposal · team rule · work item · trace

Everything else (belief conflicts, memory utility, governance internals) is expert-surface only. **[built]**

The loop, and how each step serves a persona:

capture a source	→ engineer/founder stops hunting for context
ask it privately (cited)	→ private answer; first-brain-first; no leak
choose what to share	→ share proposal (source-checked, redacted)
review	→ reviewer verifies source + impact; audited
team rule / decision	→ studio judgment becomes reusable
agent context pack + handoff	→ agent gets scoped, governed context
trace	→ what the agent read, did, proposed; who approved
reviewed learning	→ the brain improves — only after review

The chain a buyer pays for:

source → scoped context pack → human/agent work → trace → review
→ accepted brain/skill/rule update → outcome learning

Positioning. Artemis is the governed brain/control layer **above** the agent runtimes — runtime-agnostic, exporting governed context to Claude Code / Cursor / MCP. It is *not* enterprise search, *not* a Notion/Slack clone, *not* a graph visualization toy, and *not* an autonomous actor.

8. Key workflows & acceptance criteria

User stories with **observable** acceptance criteria ("done" = a behavior you can watch, not a feature that merely exists).

WF-1 — Add a source and ask it privately [built]

As an engineer, I add a repo doc / client note / PDF and ask a question, so I get a grounded answer without interrupting anyone.

- Uploading a supported file (PDF/DOCX/text/URL) creates a real, private source.
- Asking returns an answer **with citations to source spans**; an unanswerable question says so instead of fabricating.
- The source and answer stay **private until the user chooses to share**.
- Prompt-injection text inside a source does not change system behavior.

WF-2 — Choose what to share, and review it [built]

As a reviewer, I turn a useful private answer into a proposed team lesson and gate it, so only verified knowledge becomes team truth.

- "Choose what to share" creates a pending proposal with source citation, sensitivity, and redaction state.
- **Accept is blocked until the source is checked.** Reject does not mutate the Company Brain.
- The reviewer sees whether the claim is AI-drafted, human-asserted, or source-backed.

WF-3 — Make a team rule / decision [built]

As a founder, I turn a reviewed lesson into an operating rule the studio and its agents follow.

- A rule cannot become active without accepted evidence.
- Missing tradeoff / owner / next step blocks the commit.
- The rule links to the work it creates.

WF-4 — Prepare governed agent context & hand off work [built]

As an engineer, I send an agent a task with exactly the context it's allowed to see.

- The handoff shows objective, allowed context, **denied context (with reasons)**, skill, write mode, budget, and stop condition, and requires approval before running.
- Denied/private/pending context never appears in the pack.
- Attempting a run without an approved preflight is blocked and audited.

WF-5 — Read the trace, then decide what to learn [built]

As a reviewer, I see what the agent actually did and decide what (if anything) becomes memory.

- The trace shows context used, context denied, actions/cost, safety blocks, proposed writes, and approval state.
- Agent output **cannot auto-learn**; proposed learning requires explicit review.
- Harmful/low-quality memory can be quarantined from future packs.

WF-6 — Complete the loop without a guide [hypothesis]

As a first-time user, I go capture → ask → share → rule → handoff → trace → save-lesson and can explain each step aloud.

- No internal jargon on beginner surfaces; one obvious next action per screen.
 - A new user can answer: what is this, what do I do first, what becomes private / shared / agent-visible? *(This is the primary thing validation must confirm.)*
-

9. Requirements

9.1 Functional (business-level, not implementation)

#	Requirement	Priority	Status
F-1	Ingest studio sources (repo docs, client notes, PDFs, URLs, pasted text) into private, per-user sources.	Must	[built]
F-2	Answer questions over selected sources with citations; refuse when unsupported.	Must	[built]
F-3	Promote a private insight into a team proposal with source, redaction, sensitivity.	Must	[built]
F-4	Human review gate: accept requires a source check; reject never mutates canonical memory.	Must	[built]
F-5	Team rules with evidence, tradeoff, owner, review date, and linked work.	Must	[built]
F-6	Governed Agent Context Pack: allowed + denied context (with reasons), skill, constraints; exportable to Claude Code / Cursor / MCP.	Must	[built]
F-7	Worker preflight before execution: objective, scope, budget, stop condition, approval.	Must	[built]
F-8	Trace of agent reads, actions, cost, proposed writes, approvals.	Must	[built]
F-9	Governed updater: maker proposes, checker verifies, human approves high-impact writes.	Must	[built]
F-10	Measurement: cost per useful loop, repeated-explanation reduction, context-pack reuse.	Must	[built] instrumentation; [hypothesis] the metrics improve
F-11	Beginner surface exposing ~6 objects; advanced objects hidden, not renamed.	Must	[built]
F-12	Do-not-repeat / reusable-skill memory for agents.	Should	[hypothesis]
F-13	Onboarding view: current beliefs, decisions, first tasks with sources.	Should	[hypothesis]

9.2 Non-functional

#	Requirement	Priority	Status
N-1	Personal/company isolation and tenant isolation enforced in the backend (a UI cannot leak across the boundary).	Must	[built]
N-2	Governance is enforced server-side, never faked by the client.	Must	[built]
N-3	Secrets/credentials redacted or blocked; prompt injection treated as hostile data.	Must	[built]
N-4	Restricted context never leaks into answers, packs, or traces; denials are inspectable.	Must	[built]
N-5	Immutable audit of promotions, approvals, and agent actions.	Must	[built]
N-6	Model cost tracked per call; caching / bring-your-own-key path for cost control.	Must	[built] telemetry; [hypothesis] caching wired end-to-end
N-7	Fast capture; heavy parsing/indexing runs async; bounded (not document-flooding) retrieval.	Should	[built]

10. Constraints & hard product rules

Business constraints

These are the real-world limits the product has to live inside. Think of them like the runtime environment: ignore them and the thing crashes no matter how good the code is.

- **Who we can actually reach.** Our fastest path to first customers is the MorcaLabs / Stellar / Web3 network — people we can already get a meeting with. The pain isn't Web3-specific, but you sell where you have a door. **[hypothesis]**
- **Who the product is *not* for.** Only studios *already* using Claude Code / Cursor / MCP. Not generic startups, not small businesses, not big enterprise, not local services. If they're not already living inside coding agents, the value prop doesn't land. Saying no to those is a feature, not a gap. **[hypothesis]**
- **The margin trap.** Every answer and every governed loop run costs us real money in model tokens. It's easy to sign customers, look successful, and quietly lose money on every one because the model spend eats the revenue. So we measure *cost per useful loop* from day one — same discipline as watching your cloud bill before it 10x's. **[built] to measure**
- **Who we're up against.** The big players (Glean, ~\$300M ARR; Dust, \$40M Series B) are real but aimed at large enterprises, not small studios. We don't beat them by being a smaller version of the

same thing — we win by being studio-shaped and by owning the governance/audit layer they treat as a checkbox. **[validated]** (market scan, July 2026)

Hard product rules (non-negotiable)

1. **First Brain before Second Brain** — AI drafts, a human asserts, a reviewer verifies, *then* the brain mutates. High-impact memory cannot be AI-only.
 2. **Trust-gated capture** — no invisible meeting/chat recording; manual/imported/ editable capture first, connectors later, always reviewable.
 3. **Governed promotion** — uploading, indexing, or asking a source does **not** make it company truth. Only explicit review does.
 4. **Memory is reference, not command** — learned memory enters agent context as reference; it becomes a rule/constraint only after approval; suspicious memory is quarantined.
 5. **Governance is server-side** — every rule that decides what becomes team knowledge is enforced in the backend, regardless of any UI affordance.
-

11. Business model & go-to-market

This is the part that's furthest from writing code, so here it is in plain terms: how the product makes money, why it keeps making money, and how we get the first customers.

- **What we charge for.** Not "seats" alone and not raw AI usage — we bill for *governed work getting done*: an agent context pack produced, a governed loop run completed, an agent's output checked, a brain update accepted, a work handoff delivered. Think of these as the billable "useful units" — the equivalent of charging per successful job, not per API call. Right now we just *measure* them; pricing comes after we know what a unit costs and what it's worth. **[built] to measure**
 - **How pricing will probably work.** A low flat per-person fee so the bill is predictable, plus a bucket of included governed units, plus a "bring your own API key" option for teams that burn a lot of model spend (so their heavy usage doesn't wreck our margin). No public prices until we've seen real cost-per-loop numbers from actual pilots. **[hypothesis]**
 - **Why it stays defensible (the moat), in one line:** anyone can copy the *feature* — a review step, a context pack. Nobody can copy the *data we accumulate*: the growing, audited record of context-used → agent-action → human-decision → outcome, which only exists because real teams ran real governed work through us over time. The more a studio's decisions, rules, and skills live in Arteamis, the more it becomes their system of record and the harder it is to rip out. **[hypothesis]** — we still have to prove that dataset actually makes future agent work better.
 - **How we land the first customers.** Founder-led and hands-on: we personally build one studio's Studio Brain + context pack, show them it saved real time, then turn what worked into product. Concierge first, automation later — not a self-serve signup wall on day one. **[hypothesis]**
-

12. Success metrics

Grouped by what they prove. Targets are hypotheses until pilots run.

- **Product works end-to-end** — a studio completes capture → ... → saved lesson on real content, with a shipped artifact and a full audit trail. Target: 1 golden case before broad claims. **[built] path; [hypothesis] on a real studio**
 - **Adoption / value** — repeated-explanation reduction $\geq 50\%$ in a successful pilot; agent context packs reused across ≥ 3 tasks; ≥ 1 accepted brain update per active project per week. **[hypothesis]**
 - **Governance safety** — source-backed accepted updates $\geq 95\%$; restricted-context leakage = 0; rejected updates mutating canonical brain = 0. **[built] to enforce; [validated] on the security evals**
 - **Commercial proof** — 3–5 paying design-partner studios before broad GTM; cost per useful loop tracked per project/model; gross-margin visibility per customer. **[hypothesis]**
 - **Comprehension** — a new user completes the loop and explains it aloud. **[hypothesis]** — the single most important early signal.
-

13. Risks, open questions & validation plan

Top risks

Risk	Why it matters	Mitigation
Wedge too abstract	"Company brain" inspires but may not own a budget line.	Sell "stop re-explaining context to your agents"; prove time saved.
No real-user validation yet	Everything user/market-facing is a hypothesis.	The validation plan below is the top priority, above more building.
Governance is copyable	Incumbents can add a review step.	Compound the reviewed-outcome dataset; own the studio system-of-record.
Cost surprise	Margin can collapse under frontier-model context.	Cost per loop visible from day one; caching / bring-your-own-key.
Comprehension failure	A correct backend with a confusing UI still fails.	Simplified beginner surface (built); test it with real users.
Trust / adoption	Studios may resist "recording" knowledge.	Private-by-default, explicit promotion, no hidden capture, audit.

Open questions (validation must answer)

1. Will a studio CTO pay for this, and how much? (per-seat vs. per-loop)
2. Which single studio workflow proves value fastest?
3. Does a context pack measurably reduce repeated explanation and redone work?

4. Is Web3 the right beachhead, or just the fastest door?
5. What's the minimum Studio Brain a design partner will actually use daily?
6. Which agent runtime should be the first governed adapter?

Validation plan (the actual next step)

The next proof is **not** another PRD. It is real conversations and one real Studio Brain.

1. **~15 buyer conversations** with studio CTOs/founders already using coding agents. Goal: confirm (or kill) the pain, the wording, and willingness to pay.
2. **One hand-built Studio Brain + Agent Context Pack** on a real design-partner project (concierge). Goal: observe the loop with a real user; measure repeated-explanation reduction and cost per useful loop.
3. **Update this PRD from what's observed** — flip [hypothesis] tags to [validated] or cut them; capture new ideas separately rather than expanding this document.

When those three are done, this stops being a set of hypotheses and becomes a validated product spec. That — not more writing — is what makes it the best PRD it can be.

Sources & evidence base

The full research base behind Arteamis, organized by category. The **load-bearing** set is the handful the thesis actually rests on; the rest is supporting market, research, and design context.

Load-bearing — the thesis rests on these

- [YC Requests for Startups](#) — the origin problem: company knowledge is fragmented and non-executable; agents need machine-native context. (Company Brain / AI-OS / Software-for-Agents.)
- [MIT GenAI Divide](#) — 95% of enterprise GenAI pilots show no P&L impact when bolted onto old workflows.
- [WORKBank / Human Agency Scale](#) — automation vs. augmentation is per-task; every workflow declares a human-agency level.
- [Agentic AI deployment barriers](#) — verification gaps, context limits, non-determinism, confidentiality as real production blockers.
- [Blueprint First, Model Second](#) — separate workflow blueprint from probabilistic execution.
- [Pega Blueprint](#) — workflow-first, SLA-bound agent governance is already a productized enterprise pattern.
- [Glean crosses \\$300M ARR](#) — enterprise work-AI/search is large and incumbent-owned.
- [Dust Series B · Sifted coverage](#) — multiplayer agent/context platforms are funded and adopted.
- [Sentra seed](#) — a funded competitor chasing company memory from the enterprise-aggregation path.

- [OpenAI Company Knowledge](#) — model providers connect company context natively; generic retrieval is not a moat.
- [Anthropic Agent Skills](#) — skill bundles are now an open platform primitive; differentiate on provenance/approval/outcome-learning.
- [Memory injection attacks](#) · [Misevolution in self-evolving agents](#) — the safety case for "memory as reference, not command," with maker/checker review before mutation.

Market framing & failure patterns

- [AWS: AI-Driven Development Life Cycle](#) — AI work needs clarification, human validation, execution, accumulated context.
- [ITPro: Outcome-as-Agentic-Solution \(OaaS\)](#) — enterprise AI shifting from tool access to accountable outcomes.
- [Lean Startup](#) · [Customer Development](#) — build-measure-learn; test assumptions outside the building.
- [DMAIC](#) · [Goodhart's Law](#) — root-cause diagnosis over vibes; metrics become theater once targeted.
- Failure patterns: [Quibi](#) · [Bird](#) · [hardware demand traps](#) — capital/activity can't rescue a wrong input, broken economics, or unwanted output.
- [AllTech: What Is A Company Brain?](#) — capture must happen where work happens.
- Nano Thoughts: [Company Brain](#) · [System 3](#) · [Enterprise General Intelligence](#) — memory needs structure/reflection/reuse; the promotion path as the multiplayer engine; EGI as north star only.
- [Ross Dawson: BCI companies](#) — reducing friction between intent and machine action.

Competitors & market signals

- Microsoft Build 2026: [main](#) · [Windows dev platform](#) · [platform security for AI agents](#) — OS-enforced agent containment/identity/policy.
- Notion: [MCP](#) · [Custom Agents](#) · [Developer Platform](#) — workspace-as-agent-surface.
- [Eudia Series A \(\\$105M\)](#) — company-brain-like systems win via verticals (legal).
- [Workday completes Sana acquisition](#) — AI knowledge/workflow moving into enterprise suites.
- Perplexity Computer: [product](#) · [What is Computer?](#) · [for Enterprise](#) · [Computer Skills](#) · [self-improving memory](#) — async multi-step AI workers.
- Commoditizing "company-brain" landing-page tier (aggregate signal, not individual threats): [Quelvio](#) · [Teamkit](#) · [ZaliBrain](#) · [Centraligence](#) · [Inherent.sh](#) (*closest positioning overlap; watch-item*) · [Company-Brain.ai](#) · [Areev](#) (*pivoted away from company-brain*).

OSS adopted or benchmarked

- **Adopt now:** [FastMCP](#) · [Docling](#) · [Presidio](#) · [LangGraph](#).
- **Source-workspace / notebook patterns:** [Open Notebook](#) · [Vercel Knowledge Agent Template \(architecture\)](#) · [claude-obsidian](#) · [Rowboat](#).
- **Portable / agent memory:** [EverOS](#) · [Mem0](#) · [Letta](#) · [LangMem](#) · [projectmem](#).
- **Graph / vector / RAG:** [Graphiti](#) · [Cognee](#) · [Qdrant](#) · [Vespa](#) · [Microsoft GraphRAG](#) · [LightRAG](#).

- **Jobs / infra / auth / secrets:** [Temporal](#) · [Hatchet](#) · [Prefect](#) · [Zitadel](#) · [OpenFGA](#) · [TruffleHog](#) · [Gitleaks](#).
- **Agent/skill pack pattern:** [agency-agents](#) · [agency-agents-app](#) · [Agent Skills open format](#).

Research papers

- [Why Early-Stage Startups Fail](#) · [Reflexion](#) · [LongMemEval](#) · [Zep/Graphiti temporal memory](#) · [A-MEM](#)
- [Mem0 long-term memory](#) · [HumanAgencyBench](#)
- [Portable Agent Memory](#) · [AMP / memorywire](#) · [PROJECTMEM](#) — vendor-neutral, provenance-preserving, portable agent memory.
- Browser/PWA boundary risk: [User Profiles: Achilles' Heel of Browsers](#) · [Demystifying PWA Permissions](#).

UX & design

- [NN/g: Progressive Disclosure](#) · [Trevor Calabro: Onboarding & In-App Help](#) · [AI Positive: Progressive Disclosure Matters](#) — show what's needed first (the discipline behind the beginner/expert surfaces).

Pricing & infrastructure

- [AWS S3](#) · [Cloudflare R2](#) · [Qdrant Cloud](#) — storage is cheap; indexing/search/retrieval is the real cost.
- [OpenAI API pricing](#) · [Anthropic prompt caching](#) — model-cost + caching mechanics.
- [n8n pricing](#) (OSS + hosted usage) · [Devin pricing](#) (agent work as work-units, not seats).

Practitioner & community

- [Claude + Obsidian second brain](#) · [100% local AI second brain \(Rowboat\)](#) · [The New AI Order: I Built A Company Brain](#).
- Perplexity Computer practitioner guides: [Karo Zieminski](#) · [Aakash Gupta](#) · [for PMs](#) · [Department of Product](#) — the vague-task → runaway-cost failure mode (the case for task preflight contracts).

Security & platform boundary

- [MDN File System API](#) · [Chrome File System Access](#) — local files need explicit permission.
- [Tauri v2 security](#) · [Electron security](#) — narrow native bridge for a future desktop shell.
- [OWASP MCP Top 10](#) — tool poisoning, injection, missing audit, context over-sharing — direct input to the governance/audit/scoped-context design.